Track:	**DS-02**
Name:	Nikita Tiurkov
Email:	n.tiurkov@innopolis.university

Assignment 2 Introduction to Big Data

Methodology

In this section I describe the architecture of my search engine (Indexing and Ranking).

Data preparation (PySpark)

During this stage I prepare the raw dataset for indexing using PySpark and store it in HDFS.

First of all, my pipeline transfers `a.parquet`, which contains raw data from Kaggle, to HDFS. Secondly, I run `prepare_data.py` as a Spark job (in `prepare_data.sh`) on the master node. This script performs the following operations:

1. Reads the parquet file and selects only the `id`, `title`, and `text` columns;
2. Samples 5% of texts from the dataset*;
3. Filters out unacceptable texts (empty, HTML, XML, JSON) and takes only 1000 texts;
4. Writes the texts to the local filesystem in the `data` directory;
5. Moves the data to HDFS from the local filesystem;
6. Parses the texts and saves a TSV file for later indexing into `/input/data` folder in HDFS.

* Note that ideally I should filter out unacceptable texts first and only then sample 1000 texts. But such an approach would require processing (with `is_acceptable_plain_text`) the whole dataset, which isn't feasible for the current assignment. Though, as the raw dataset contains around 250k texts, I could assume that sampling 5% of texts and filtering later is reliable enough for the purpose of the assignment.

Index Creation (Hadoop MapReduce)

During this stage I build an inverted index using Hadoop MapReduce (`create_index.sh`) and store it in HDFS.

My MapReduce pipeline consists of two stages:

1. Build an inverted index* from raw texts and compute term statistics (`mapper1.py` and `reducer1.py`):
   - **Input**: Raw data in HDFS (`/input/data`) in (doc_id, title, text) format;
   - **Output**: Inverted index in HDFS (`/indexer/index`) in (term, doc_id, tf, df) format;
   - **Mapper**: Reads raw texts from HDFS, lowercases and extracts only alphanumeric tokens. Then emits (term, doc_id, 1) tuples, where term is used as a key for grouping by the reducer;
   - **Reducer**: Groups all occurrences of the same term and emits (term, doc_id, tf, df) tuples, where tf is the term frequency and df is the document frequency.
   - **Purpose**: Build an inverted index for fast lookups and compute term-level statistics, required for BM25 ranking.
2. Compute document and corpus statistics (`mapper2.py` and `reducer2.py`):
   - **Input**: Raw data in HDFS (`/input/data`) in (doc_id, title, text) format;

- **Output**: Document and corpus statistics in HDFS (`/indexer/stats`) in (doc_id, length) and (**GLOBAL**, num_docs, avg_doc_len) format;
- **Mapper**: Reads raw texts from HDFS, lowercases and extracts only alphanumeric tokens. Then emits (doc_id, length) tuples, where doc_id is used as a key for grouping by the reducer and length is the number of tokens in the document;
- **Reducer**: Groups all occurrences of the same document (key) and emits (doc_id, length) tuples (as all occurrences for the same document contain the same lengths, it takes the first one). Moreover, it computes the average length of documents in the corpus and emits it as (**GLOBAL**, num_docs, avg_doc_len) tuple.
- **Purpose**: Compute document-level and corpus-level statistics, required for BM25 ranking.

* Note that I do not create a separate stage for vocabulary construction as this stage seems redundant for me. In my architecture the inverted index is built to contain both postings and term statistics for a given term. Hence, vocabulary lookup (to check if a term is present in the vocabulary), posting retrieval and term statistics retrieval can be performed in the same stage.

Index Storing (PySpark + Cassandra)

During this stage I move the constructed inverted index and statistics from HDFS to Cassandra (`store_index.sh` and `load_index_to_cassandra.py`). Note that my architecture is designed for full reindexing. So, each run of `store_index.sh` rewrites data in Cassandra with new index.

To transfer data between HDFS and Cassandra I use PySpark with Cassandra Connector. Such an approach is more scalable than using native bulk insertions with `cassandra-driver` . Hence, I use `cassandra-driver` only to perform DDL operations in Cassandra.

For efficient retrieval I use the following schema:

1. `inverted_index` table*:
   - **Partition key**: `term` ;
   - **Clustering columns**: `doc_id` ;
   - **Columns**: `tf` , `df` ;
   - **Purpose**: Store an inverted index for fast lookups of postings and term statistics for a given term. It also allows one to efficiently retrieve all postings and term statistics for a given term in one read.
2. `doc_stats` table:
   - **Partition key**: `doc_id` ;
   - **Columns**: `title` , `length` ;
   - **Purpose**: Store document-level statistics for a given document.
3. `corpus_stats` table:
   - **Partition key**: `id` (single row with `id` = `global`);
   - **Columns**: `num_docs` , `avg_doc_len` ;
   - **Purpose**: Store corpus-level statistics for the entire corpus.

* As already mentioned above, my inverted index is designed to be both an inverted index (for postings lookup) and a vocabulary (for term presence check and term-level statistics retrieval). I think that creating a separate table for vocabulary is redundant as (1) it won't be used in this assignment and (2) my inverted index could sufficiently perform all the necessary operations.

Ranking Engine (PySpark)

After indexing I have all data stored in Cassandra ready to be used for ranking. Ranking uses PySpark (`search.sh` and `query.py`) and is logically separated into three stages:

1. Fetch index data from Cassandra:
   - **Input**: Query terms (from the user);
   - **Output**: PySpark DataFrames with postings, term/document/corpus statistics;
   - **Purpose**: Load index data from Cassandra into a PySpark DataFrame;

2. Compute BM25 scores for each document:
   - **Input**: PySpark DataFrames with postings, term/document/corpus statistics;
   - **Output**: a PySpark DataFrame with BM25 scores for each document;
   - **Purpose**: Compute BM25 scores for each document for the given query;
3. Retrieve top N documents:
   - **Input**: a PySpark DataFrame with BM25 scores for each document;
   - **Output**: List of top N documents with their BM25 scores;
   - **Purpose**: Retrieve top N documents from a PySpark DataFrame and print them to the console;

Rerun Consistency

For demonstration purposes I design `*.sh` and `*.py` files to allow rerunning the `app.sh` script without re-creating the containers. This includes:

1. `start-services.sh` script initially stops running services if they are currently running and only then restarts them. This allows one to avoid errors connected with starting Hadoop services multiple times;
2. `load_index_to_cassandra.py` script truncates all tables in Cassandra before loading new data to avoid data duplication on reruns.
3. `index.sh` script cleans up HDFS temporary files (after loading them to Cassandra) to avoid data duplication on reruns.

Demonstration

In this section I explain how to run my search engine and demonstrate that it works as expected.

Running the search engine

Step 1: Install prerequisites

- Docker
- Docker Compose

Step 2: Clone the repository

```
git clone https://github.com/KosmonikOS/big-data-assignment2
```

Step 3: Download the dataset

For the purpose of the assignment I use the specified [dataset from Kaggle](). Before proceeding to the next step, please download it and place the `a.parquet` file in the `/app` directory on the local machine.

Step 4: Run the containers

```
docker compose up
```

This will create 3 containers (a master and a worker node for Hadoop, and a Cassandra server). The master node will run the script `app/app.sh` as an entrypoint (it will index the data and run the example search query).

Step 5: Run the retrieval

After the indexing is complete, you can run the retrieval by executing the following commands:

```
docker exec -it cluster-master bash
chmod +x search.sh
./search.sh "This is my query!"
```

Verification

I take all screenshots from the Portainer GUI that allows you to easily manage docker containers on the remote machine.

Below is the screenshot showing the successful completion of the indexing process. Note that the full docker container log doesn't fit into the console, so I download it as txt and open it on the right of the screenshot.

Not Secure — 45.92.176.125

BS - Assignment... | KosmonikOS/big-... | apache/cassandra... | Wikipedia Plaintex... | Portainer | local

Contains | Search | Done

Log viewer settings

Auto-refresh logs

Wrap lines

Display timestamps

Fetch: All logs

Search: Filter...

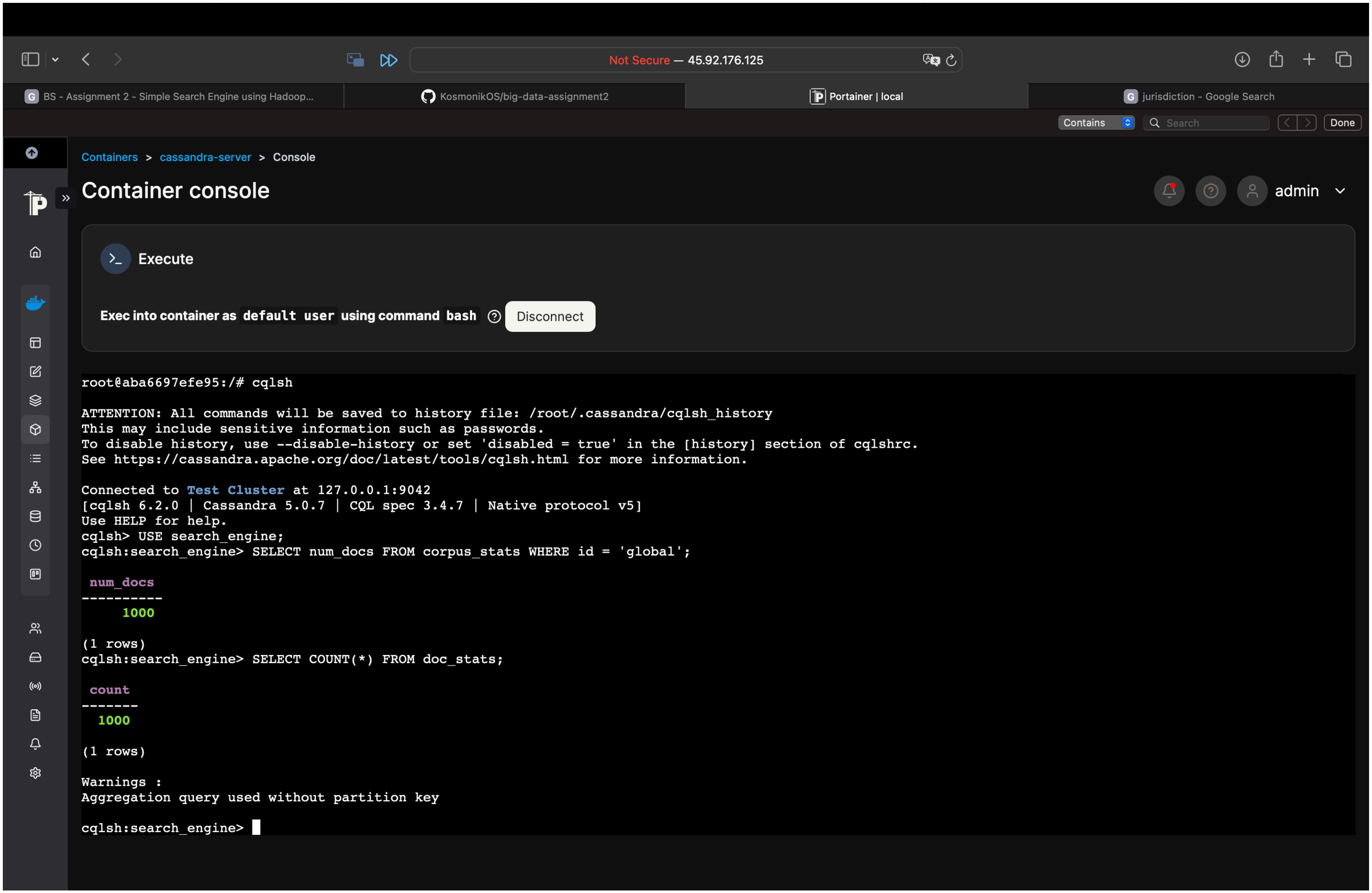
Lines: 100

Actions: Download logs | Copy | Copy selected lines | × Unselect

```
avoid this, it is best practice to explicitly set Cluster(protocol_version) to the version suppor
ted by your cluster. https://docs.datastax.com/en/developer/python-driver/latest/api/cassandra/cl
uster.html#cassandra.cluster.Cluster.protocol_version
WARNING:cassandra.cluster:Downgrading core protocol version from 65 to 5 for 172.18.0.3:9042. To
avoid this, it is best practice to explicitly set Cluster(protocol_version) to the version suppor
ted by your cluster. https://docs.datastax.com/en/developer/python-driver/latest/api/cassandra/cl
uster.html#cassandra.cluster.Cluster.protocol_version
INFO:cassandra.policies:Using datacenter 'datacenter1' for DCAwareRoundRobinPolicy (via host '17
2.18.0.3:9042'); if incorrect, please specify a local_dc to the constructor, or limit contact poi
nts to local cluster nodes
INFO:__main__:Schema created.
INFO:__main__:Tables truncated.
INFO:__main__:Loading inverted_index from hdfs://cluster-master:9000/indexer/index
INFO:__main__:inverted_index loaded.
INFO:__main__:Loading doc_stats and corpus_stats from hdfs://cluster-master:9000/indexer/stats
INFO:__main__:doc_stats loaded.
INFO:__main__:corpus_stats loaded.
INFO:__main__:All tables loaded successfully.

>> Clearning HDFS temporary files
Deleted /indexer
Deleted /input/data
Deleted /data
Deleted /a.parquet
```

```
[#################################        ] | 80% Completed | 18.2s
[#################################        ] | 81% Completed | 18.3s
[##################################       ] | 83% Completed | 18.4s
[###################################      ] | 85% Completed | 18.5s
[####################################     ] | 87% Completed | 18.6s
[#####################################    ] | 89% Completed | 18.7s
[######################################   ] | 92% Completed | 18.8s
[#######################################  ] | 97% Completed | 18.9s
[######################################## ] | 99% Completed | 19.0s
[########################################] | 100% Completed | 19.1s

> Preparing data for indexing

> Data is ready for indexing

> Indexing data

>> [1/2] Running MapReduce pipelines

>>> Pipeline 1: Building inverted index

>>> Pipeline 2: Computing document statistics

>> [2/2] Loading HDFS index into Cassandra

>>> Loading HDFS index into Cassandra
:: loading settings :: url = jar:file:/usr/local/spark/jars/ivy-2.5.1.jar!/org/apache/ivy/core/
settings/ivysettings.xml
INFO:__main__:Waiting for Cassandra at ['cassandra-server']
WARNING:cassandra.cluster:Cluster.__init__ called with contact_points specified, but no
load_balancing_policy. In the next major version, this will raise an error; please specify a
load-balancing policy. (contact_points = ['cassandra-server'], lbp = None)
WARNING:cassandra.cluster:Downgrading core protocol version from 66 to 65 for 172.18.0.3:9042.
To avoid this, it is best practice to explicitly set Cluster(protocol_version) to the version
supported by your cluster. https://docs.datastax.com/en/developer/python-driver/latest/api/
cassandra/cluster.html#cassandra.cluster.Cluster.protocol_version
WARNING:cassandra.cluster:Downgrading core protocol version from 65 to 5 for 172.18.0.3:9042. To
avoid this, it is best practice to explicitly set Cluster(protocol_version) to the version
supported by your cluster. https://docs.datastax.com/en/developer/python-driver/latest/api/
cassandra/cluster.html#cassandra.cluster.Cluster.protocol_version
INFO:cassandra.policies:Using datacenter 'datacenter1' for DCAwareRoundRobinPolicy (via host
'172.18.0.3:9042'); if incorrect, please specify a local_dc to the constructor, or limit contact
points to local cluster nodes
INFO:__main__:Cassandra is ready.
WARNING:cassandra.cluster:Cluster.__init__ called with contact_points specified, but no
load_balancing_policy. In the next major version, this will raise an error; please specify a
load-balancing policy. (contact_points = ['cassandra-server'], lbp = None)
WARNING:cassandra.cluster:Downgrading core protocol version from 66 to 65 for 172.18.0.3:9042.
To avoid this, it is best practice to explicitly set Cluster(protocol_version) to the version
supported by your cluster. https://docs.datastax.com/en/developer/python-driver/latest/api/
cassandra/cluster.html#cassandra.cluster.Cluster.protocol_version
WARNING:cassandra.cluster:Downgrading core protocol version from 65 to 5 for 172.18.0.3:9042. To
avoid this, it is best practice to explicitly set Cluster(protocol_version) to the version
supported by your cluster. https://docs.datastax.com/en/developer/python-driver/latest/api/
cassandra/cluster.html#cassandra.cluster.Cluster.protocol_version
INFO:cassandra.policies:Using datacenter 'datacenter1' for DCAwareRoundRobinPolicy (via host
'172.18.0.3:9042'); if incorrect, please specify a local_dc to the constructor, or limit contact
points to local cluster nodes
INFO:__main__:Schema created.
INFO:__main__:Tables truncated.
INFO:__main__:Loading inverted_index from hdfs://cluster-master:9000/indexer/index
INFO:__main__:inverted_index loaded.
INFO:__main__:Loading doc_stats and corpus_stats from hdfs://cluster-master:9000/indexer/stats
INFO:__main__:doc_stats loaded.
INFO:__main__:corpus_stats loaded.
INFO:__main__:All tables loaded successfully.

>> Clearning HDFS temporary files
Deleted /indexer
Deleted /input/data
Deleted /data
Deleted /a.parquet
```

Below is the screenshot showing that after indexing Cassandra contains exactly 1000 documents. To show this I connect to Cassandra to get the number of documents with computed statistics.



Below are the screenshots showing 3 example queries. As I only show the titles of retrieved documents, I cannot ensure relevance of whole document texts. Though, I can observe that at least titles are connected to the query (or the general topic of the query).

Not Secure — 45.92.176.125

BS - Assignment 2 - Simple Searc... | KosmonikOS/big-data-assignment2 | apache/cassandra-spark-connect... | Wikipedia Plaintext (2023-07-01) | Portainer | local | portainer/portainer: Making Docke...

Contains | Search | Done

Upgrade to Business Edition

PORTAINER.io
COMMUNITY EDITION

- Home
- local
  - Dashboard
  - Templates
  - Stacks
  - Containers
  - Images
  - Networks
  - Volumes
  - Events
  - Host

Administration

- User-related
- Environment-related
- Registries
- Logs
- Notifications
- Settings

© Portainer Community Edition 2.39.1 LTS

Containers > cluster-master > Console

Container console

admin

Execute

Exec into container as `default user` using command `bash` Disconnect

```
root@cluster-master:/app# ./search.sh "Architecture, buildings and cities"

> Running search engine
:: loading settings :: url = jar:file:/usr/local/spark/jars/ivy-2.5.1.jar!/org/apache/ivy/core/settings/ivysettings.xml

Top 10 relevant documents:
 1. [28380942] A_&_P_Food_Stores_Building  (score=14.8117)
 2. [3537854] A_Cañiza  (score=8.7711)
 3. [552462] A_(band)  (score=6.3797)
 4. [42175589] A_Barca  (score=6.3623)
 5. [55779313] A_Battery_Shelled  (score=5.4314)
 6. [21990161] A_Case_of_Two_Cities  (score=5.2212)
 7. [36875415] A_Boy_Bringing_Bread  (score=5.1231)
 8. [25357548] A_C_Wharton  (score=4.5192)
 9. [48405771] A_Boyar_Wedding_Feast  (score=4.4882)
10. [7125736] A_Body_in_the_Bath_House  (score=4.4415)
root@cluster-master:/app#
```

Not Secure — 45.92.176.125

BS - Assignment 2 - Simpl... | KosmonikOS/big-data-assi... | apache/cassandra-spark-c... | Wikipedia Plaintext (2023-... | Portainer | local | portainer/portainer: Makin... | jurisdiction - Google Search

Contains | Search | Done

Upgrade to Business Edition

PORTAINER.io
COMMUNITY EDITION

- Home
- local
  - Dashboard
  - Templates
  - Stacks
  - Containers
  - Images
  - Networks
  - Volumes
  - Events
  - Host

Administration

- User-related
- Environment-related
- Registries
- Logs
- Notifications
- Settings

© Portainer Community Edition 2.39.1 LTS

Containers > cluster-master > Console

Container console

admin

Execute

Exec into container as `default user` using command `bash` Disconnect

```
root@cluster-master:/app# ./search.sh "Law, court and jurisdiction"

> Running search engine
:: loading settings :: url = jar:file:/usr/local/spark/jars/ivy-2.5.1.jar!/org/apache/ivy/core/settings/ivysettings.xml

Top 10 relevant documents:
 1. [62120562] A_(a_Minor)_v_Minister_for_Justice_and_Equality_and_others  (score=11.3393)
 2. [7095820] A_Brush_with_the_Law  (score=8.8338)
 3. [17490919] A_Blueprint_for_Murder  (score=7.6801)
 4. [20842172] A_Boyfriend_for_Christmas  (score=7.6651)
 5. [71358147] A_Bolt_of_Lightning_(Studio_One)  (score=6.9142)
 6. [70026449] A_B_M_Altaf_Hossain  (score=6.6465)
 7. [10078432] A_Case_for_the_Court  (score=6.1980)
 8. [5811854] A_Caress_of_Twilight  (score=6.0891)
 9. [3537854] A_Cañiza  (score=5.7104)
10. [63007399] A_Candle_in_Her_Room  (score=5.6903)
root@cluster-master:/app#
```

Not Secure — 45.92.176.125

BS - Assignment 2 -... | KosmonikOS/big-dat... | apache/cassandra-sp... | Wikipedia Plaintext (2... | Portainer | local | portainer/portainer:... | jurisdiction - Google... | kitchen - Google Sear...

Contains | Search | Done

Upgrade to Business Edition

PORTAINER.io
COMMUNITY EDITION

- Home
- local
  - Dashboard
  - Templates
  - Stacks
  - Containers
  - Images
  - Networks
  - Volumes
  - Events
  - Host

Administration

- User-related
- Environment-related
- Registries
- Logs
- Notifications
- Settings

© Portainer Community Edition 2.39.1 LTS

Containers > cluster-master > Console

Container console

admin

Execute

Exec into container as `default user` using command `bash` Disconnect

```
root@cluster-master:/app# ./search.sh "Food, cooking and kitchen"

> Running search engine
:: loading settings :: url = jar:file:/usr/local/spark/jars/ivy-2.5.1.jar!/org/apache/ivy/core/settings/ivysettings.xml

Top 10 relevant documents:
 1. [36253472] A_Bite_of_China  (score=20.1699)
 2. [46442290] A_Book_of_Mediterranean_Food  (score=16.0056)
 3. [43482213] A_Chef's_Life  (score=15.6339)
 4. [37647052] A_Chef_of_Nobunaga  (score=7.2348)
 5. [55626506] A_Candle_for_the_Devil  (score=6.8811)
 6. [60578215] A_Big_Mooncake_for_Little_Star  (score=6.6373)
 7. [7810848] A_Chef_in_Love  (score=6.4860)
 8. [36612906] A_Cassette_Tape_Culture_(Phase_One)  (score=6.3166)
 9. [36612988] A_Cassette_Tape_Culture_(Phase_Three)  (score=6.2037)
10. [45328844] A_Boss_in_the_Living_Room  (score=6.1318)
root@cluster-master:/app#
```